



React.js

vs.

Next.js

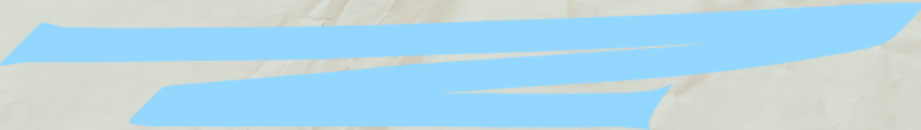


What is React.js?

React.js (or simply React) is a JavaScript library created by Facebook for building user interfaces, especially for single-page applications (SPAs). It's component-based, allowing you to break your UI into small, reusable pieces. 

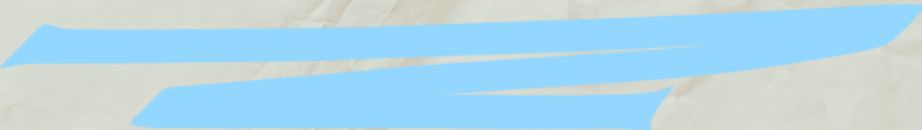


Strengths of React.js



- **Flexibility:** React is super flexible. Do whatever you want with it! 
 - **Huge Ecosystem:** The React ecosystem is massive! 
 - **Reusable Components:** Build once, reuse everywhere! 
 - **Community Support:** React has one of the largest dev communities. You'll never be alone! 
- 
- 

Weaknesses of React.js



- **Configuration Hell:** React gives you a lot of freedom, but it also means you have to do a lot of setup. 

Prepare to write some config files!



- **SEO Challenges:** Since React apps are usually rendered on the client-side, search engines might struggle to index your content. 



What is Next.js?

Now, enter Next.js, the superhero framework 🦄 built on top of React by Vercel. While React is a library, Next.js is a framework, meaning it comes with a bunch of built-in features, so you can focus more on writing code and less on configuration. 💻





Strengths of Next.js

- **SSR & SSG:** Next.js shines with Server-Side Rendering (SSR) and Static Site Generation (SSG). ⚡
- **File-Based Routing:** No more installing third-party routing libraries! You simply create files in the pages/ folder. 📁
- **API Routes:** Lets you create API endpoints without leaving the project. 🏠
- **Zero Config:** Next.js takes care of the boring stuff like webpack, code-splitting, and hot reloading out of the box. 🔧



Weaknesses of Next.js ⚠

- **Less Flexible:** With great power comes great responsibility... and some restrictions. Next.js' opinionated structure might limit you if you're looking for complete freedom. ➡ 🚧
- **File Routing Learning Curve:** Though file-based routing is great, it may feel restrictive or confusing if you're used to React Router. 📁





Key Differences Between React.js & Next.js

Feature	React.js	Next.js
Rendering	Client-side by default	Server-side and static generation
Purpose	Library for building UIs	Full-stack framework built on React
Routing	You need React Router (or similar)	Built-in file-based routing
Code Splitting	Manual configuration	Automatic
SSR / SSG	Not supported out of the box	Fully supported
API Routes	Requires a separate backend	Built-in support for API routes
SEO	Limited, due to client-side rendering	Excellent, thanks to SSR and SSG
Configuration	Needs a lot of setup	Mostly zero-config
Image Optimization	DIY	Built-in component



Which One Should You Choose?

✅ Use React.js if:

- You need complete control over your project.
- You don't mind configuring stuff yourself.
- You're building a simple SPA or a project that doesn't need SEO.
- You just want to focus on UI components.



✅ Use Next.js if:

- You want built-in SSR or SSG for better SEO and performance.
- You like the idea of file-based routing and API routes.
- You're building a complex, high-performance web app.
- You don't want to deal with complex setup, Next.js does the heavy lifting for you!



Found this helpful ?



Share to help others



Save to refer to later



Follow Hadil Ben Abdallah for more
amazing content about programming